



Debre Berhan University

College of Computing

Department

of

Information Technology

Course Title: Advanced Programming

Course Code: ITec 3057

Credit : 3 Cr. Hrs / 5 ECTS (2 Hrs. Lec. & 3 Hrs. Lab)

Target group: 3rd year Information Technology

Course Manager: Yirga G.

Chapter 1:

Java File and Database Programming

Course Learning Outcomes (CLO)

At the end of the course the students will be able to:

- **CLO1:** Demonstrate Java application file access and database connectivity
- **CLO2:** Describe multithreading and asynchronous programming concepts
- **CLO3:** Develop a solution for a given problem using multithreaded and asynchronous programming
- **CLO4:** Demonstrate socket programming in java
- **CLO5:** Develop a servlet-based java web application
- **CLO6:** Collaborate and develop enterprise backed application using current java framework

➤ Overview of OO Programming with Java

- What is object orientation?
- Object-oriented programming (OOP)
- Procedural vs OOP
- OOP model
- Advantage of OOP
- OOP Features
- Objects and Class in Java
- Class declaration in Java
- Methods in Java
- Java Development Tools

Types of programming

Procedural

- Program is written as sequence of instructions to solve specific problems.

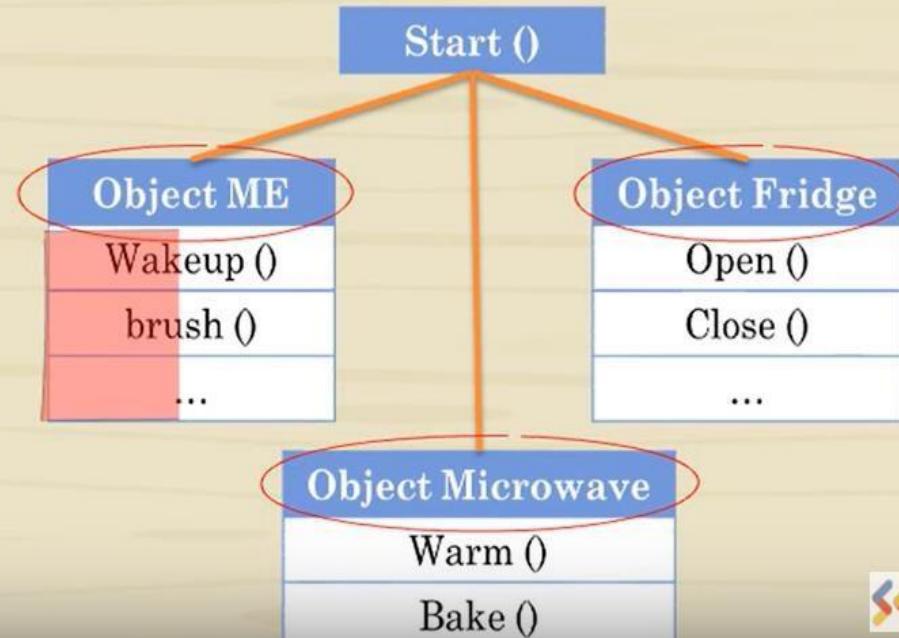
e.g.: Recipes,
Morning Steps:

Start ()
I wake up
I brush my teeth
I open fridge
I take milk
I warm it in microwave

- Top-down approach
- Code reuse is difficult

Object-oriented

- Program is an interaction of function between objects



- Code is modular
- Can be extended for reuse.

Overview of OO Programming in Java

Procedural

vs

OOP

- ◆ **Procedural programming** is a set of instructions telling a computer what to do step by step.
- ◆ **Object-oriented programming** (OOP) is a programming paradigm based on the concept of "**objects**" that contain data, in the form of fields (attributes), and code, in the form of procedures (methods).

Overview of OO Programming in Java

Procedural	OO Programming
Program is divided into small parts called functions .	Program is divided into parts called objects .
Does not have any proper way for hiding data so it is less secure .	Provides Data Hiding so provides more security .
Limited code reuse	Object focused design
Top down design	Code reuse
Example : C, VB, FORTRAN, Pascal.	Example : C++, JAVA, VB.NET, C#.NET.

The *goal of OOP methodology* is to achieve Software Systems that are **reliable**, **reusable**, **extensible**.

Overview of OO Programming in Java

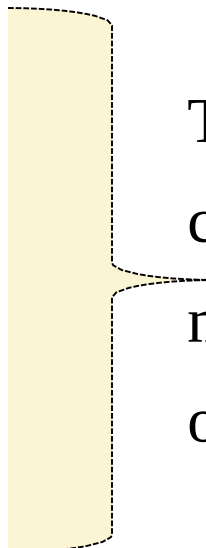
➤ What is object orientation?

- A technique for system modelling.

◆ **Object-Oriented Programming** (OOP) is a paradigm that provides many concepts such as abstraction, inheritance, encapsulation/data binding, polymorphism, etc.

◆ It simplifies the software development and maintenance by providing the concepts of:

- ◆ Objects
- ◆ Class
- ◆ Abstraction
- ◆ Inheritance
- ◆ Polymorphism
- ◆ Encapsulation



These characteristics collectively contribute to the flexibility, modularity, and maintainability of object-oriented systems.

Overview of OO Programming in Java

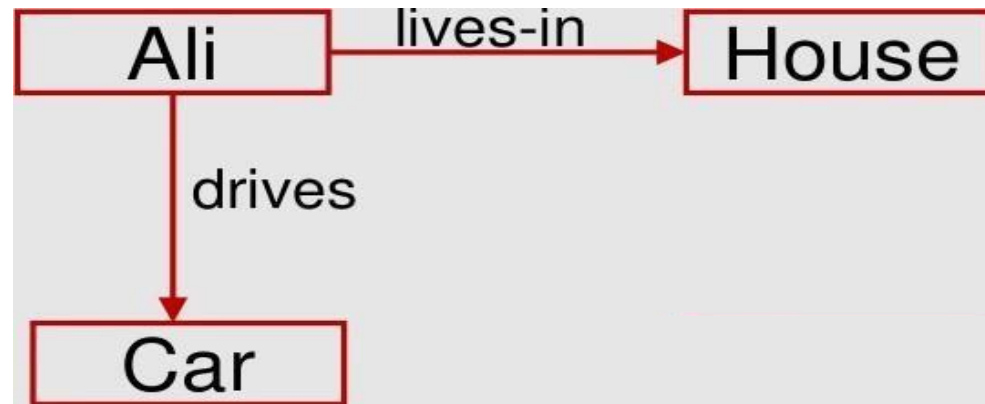
➤ What is a OO Model?

- ◆ A model is an abstraction of something.
- ◆ Purpose is to understand the product before developing it.

Examples — OO Model:

- Objects

- ✓ Ali
- ✓ House
- ✓ Car



- Interactions

- ✓ Ali lives in the house
- ✓ Ali drives the car

- ◆ OO model can consists of several interacting objects.

Overview of OO Programming in Java

➤ **Advantage of OO models**

- ◆ Objects are modeled on real world entities.
- ◆ OO enables modeling complex systems of real world into manageable software solutions.
- ◆ It provides:
 - √ data abstraction
 - √ data encapsulation
 - √ reusability of code.
 - √ easy code maintenance
- ◆ OO models are easy to develop and understand.

Overview of OO Programming in Java

➤ OOP Features

- ◆ **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects.
- ◆ It simplifies the software development and maintenance by providing the features:
 - Objects
 - Class
 - Abstraction
 - Inheritance
 - Polymorphism
 - Encapsulation

Overview of OO Programming in Java

➤ Object

- ◆ Any entity/real-world element that may have a physical existence (such as Car, Book, Person, etc.) or a conceptual existence (such as project, banking system, department, ... etc).
- ◆ The real-world objects can be defined by their *state* and their *behavior*.
 - **State:** represents data (value) of an object.
 - ✓ Descriptive characteristics – Attribute.
e.g., CAR: (Color, manufacturer, Cost, Model)
 - **Behavior:** A manner that represents how an object acts and reacts. It represents externally visible activities performed by an object in terms of changes in its state.
e.g. CAR: (Go, stop, fuel fill, ...etc.)

Overview of OO Programming in Java

➤ Class

◆ A class is set of objects (logical entity) that share common state and behavior. e.g. **class**: fruit, Employee.

◆ **An object is instance of a class.**

Example:	Class	objects
	Fruit	Apple, Banana, Mango, ...etc.
	Employee	Individual employee – Aster, Haile...

◆ A class in java contain:

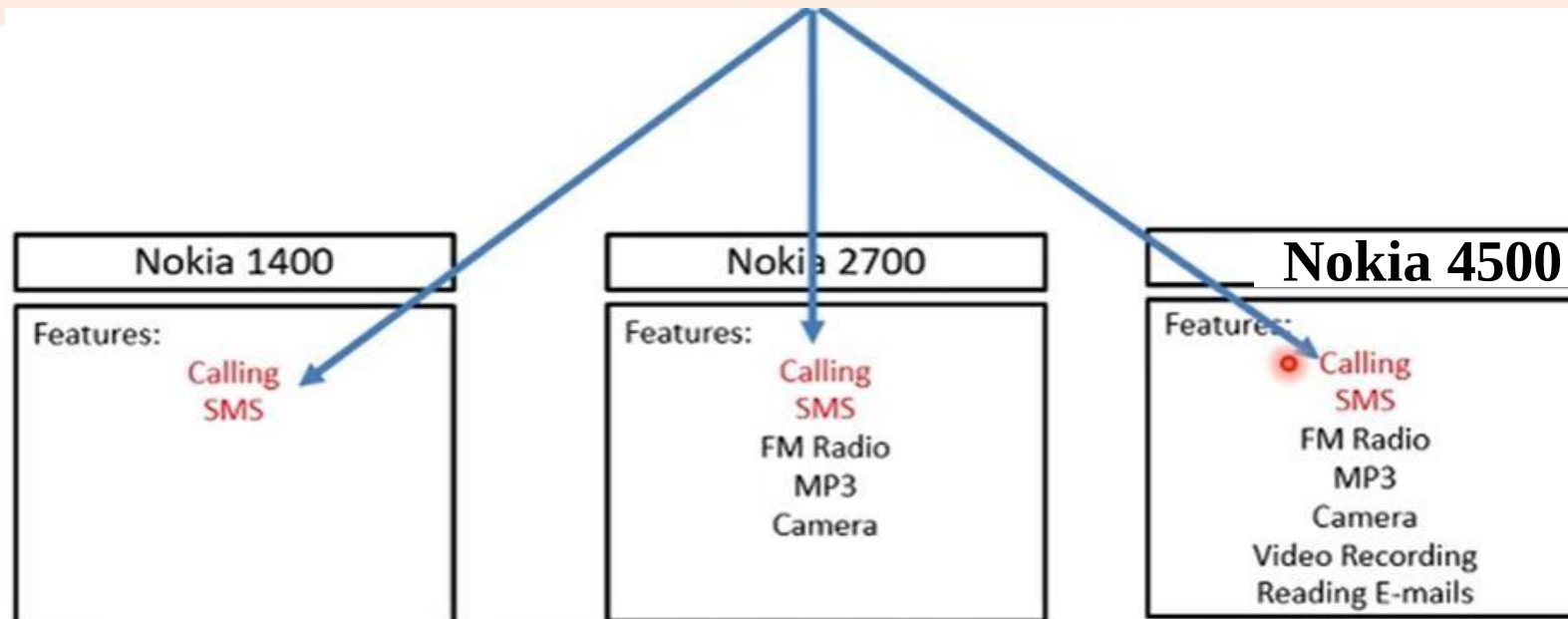
- √ Data member
- √ Method
- √ Constructor
- √ Interface

Overview of OO Programming in Java

➤ Abstraction

- Hiding internal details and showing functionality.
- In java, we use abstract class and interface to achieve abstraction.
- Its goal is to simplify the representation of complex systems by hiding unnecessary details and exposing only the essential functionalities.

e.g., Abstract information (relevant and common information) for object Mobile phone is making a **call to any number** and **send SMS**.



Overview of OO Programming in Java

➤ Inheritance

- It is a process by which Sub-class acquires all the properties and operations of Super class.
- It allows classes to share and reuse behaviors and attributes.

➤ Polymorphism: -

- the same operation may behave differently on different classes.
- It enables a single interface to represent different types of objects, providing flexibility and extensibility.

➤ Encapsulation

- Binding data and methods into single unit, called **class**.
- It is a protection mechanism by using access specifiers such as **public**, **protected** and **private**.
- **Its goal is** to secure data and functions within a class, preventing unauthorized access and modification.

Overview of OO Programming in Java

➤ Class declaration in Java

- ◆ Declaration of class must start with the keyword **class** followed by the class name and class members are declared within braces.

Example:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

Overview of OO Programming in Java

➤ **Methods in Java**

- ◆ A **method** is a group of statements that together perform a specific task.
- ◆ Every Java program has at least one function, which is `main( )`.

➤ **Why use Methods?**

- ◆ Methods help you modularize a program by separating its tasks into self-contained units.
- ◆ **Methods** are used for **divide a large code into module**, due to this we can easily debug and maintain the code.
- ◆ It reduces code complexity and increasing its maintainability.

Overview of OO Programming in Java

➤ Import Declaration in java:

- ◆ A great **strength of Java** is its rich set of **predefined classes** that you can reuse rather than "reinventing the wheel".
- ◆ These classes are grouped into packages—named groups of related classes—and are collectively referred to as the Java class library, or the Java Application Programming Interface (Java API).

e.g., **import** java.util.Scanner; // **program uses class Scanner** This is an import declaration that helps the compiler locate a class that's used in this program.

- ◆ *All **import** declarations must appear before the first-class declaration in the file.*
- ◆ *Placing an **import** declaration **inside or after a class declaration** is a **syntax error**.*

Overview of OO Programming in Java

➤ Type of Java Classes and Methods:

- ◆ Java allows the use of both **Library classes/methods** (pre-defined methods) and **User-defined methods**.
- ◆ To use the library class and methods, we "**import**" the packages (or classes individually). e.g. `java.util.Scanner`
- ◆ The **java.util package** contains a class named **Scanner** that is used to receive input of primitive types like **int**, **double**, etc., as well as a reference type such as **strings**.

for example, `Scanner input = new Scanner( System.in );`

- ◆ This creates a Scanner object named ***input*** that is associated with the standard input stream (`System.in`), allowing you to read user input from the console.

Overview of OO Programming in Java

Example:

```
// java program which read any two numbers from the user and
//compute the sum
import java.util.Scanner;
public class Addition {
    public static void main(String[] args)
    {
        Scanner input = new Scanner( System.in );
        int num1, num2,sum;
        System.out.println("Enter the first number" );
        num1 = input.nextInt();
        System.out.println("Enter the second number" );
        num2 = input.nextInt();
        sum = num1+num2; // somput the sum of the two numbers
        System.out.println(" The Sum is: " + sum);
    }
} // end class Addition
```

Overview of OO Programming in Java

➤ **Advantage of Methods:**

- ◆ Code optimization: **avoid repetition of codes.**
- ◆ Develop an application in module format.
- ◆ Reduce chances of error. i.e., Makes unit testing possible.
- ◆ Make programs easier to modify and understand.
- ◆ Can be called several times in the same program, **allowing the code to be reused.**
- ◆ A method that performs one task is easier to test and debug than one that performs many tasks.

Overview of OO Programming in Java

➤ User defined methods:

- ◆ These Methods are created by programmer according to their requirement.

e.g.,

```
public class Test {  
    public static void main(String [ ] args) {  
        int z ; //Declaration  
        z= addition( 5, 8 ) ;  
        System.out.println( " The sum is " +z ) ;  
    }  
    public static int addition(int x, int y) {  
        return x+y ;  
    }  
}
```

Library method

Calling method

User defined method

➤ **Java Development Tools**

➤ Three major development tools are:

- NetBeans: Open Source by Sun
- Eclipse: Open Source by IBM
- Jbuilder: Developed by Borland

➤ Other useful tools are:

- Code Warrior: by Metrowerks
- TextPad Editor
- Jcreator LE
- JEdit
- JGrasp
- BlueJ
- DrJava

Introduction to Collections and Class Array

- ◆ The Java API provides several predefined data structures, called **collections**, used to store groups of related objects.
- ◆ These classes provide efficient methods that organize, store and retrieve your data without requiring knowledge of how the data is being stored. This reduces application-development time.
- ◆ We have used arrays to store sequences of objects.
- ◆ **Array objects are data structures** consisting of **related data items** of the same type.

Primitive Types vs. Reference Types

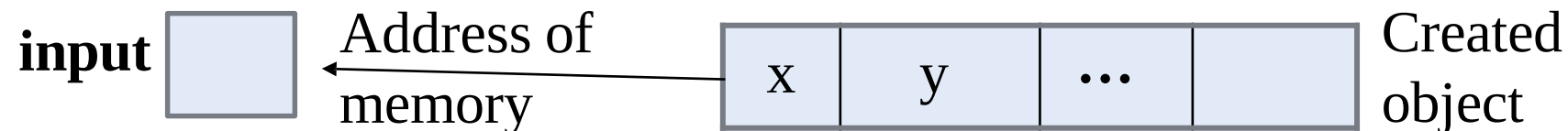
- ◆ Java's types are divided into **primitive types** and **reference types**.
- ◆ We may work with primarily with variables of types *int*, *Boolean*, *byte*, *char*, *short*, *long*, *float* or *double*.
- ◆ A **primitive-type** variable can **hold exactly one value** of its declared type at a time.

For example, **int** x ; x  Created memory

- ✓ an *int variable type* can store one integer at a time.
- ✓ When another value is assigned to that variable, the new value replaces the previous one—which is lost.

Primitive Types vs. Reference Types ...cont'd

- ◆ Primitive-type instance variables are initialized by default.
 - ✓ Variables of types *byte*, *char*, *short*, *int*, *long*, *float* and *double* are initialized to **0**, and variables of type *Boolean* are initialized to **false**.
- ◆ Programs use variables of **reference types** to store the addresses of objects in the computer's memory.
 - i.e., classes, which specify the types of objects, are reference types.
 - For example, we may use variables of the **reference types** *Scanner*.
- ◆ Objects that are referenced may each contain many instance variables. For example, *Scanner* `input = new Scanner (system.in);`
input.x, input.y, ... etc.



Primitive Types vs. Reference Types ...cont'd

- ◆ **Reference-type instance variables** are initialized by default to the **value null**, which represents reference to nothing.
- ◆ Recall that local variables are not initialized by default.
- ◆ An attempt to use an uninitialized local variable causes a compilation error.

For example,

- public class Example {
- int memberVar; // Default initialized to 0 public static void main(String[] args) {
 - Example obj = new Example();
 - int x;
 - System.out.println(obj.memberVar); // Prints: 0
 - System.out.println(obj); // compilation error.

Arrays in java

➤ What is **Array in java**?

- ◆ An **array** is a **group of variables** (called elements or components) **containing values that all have the same type**.
i.e, Arrays are data structures consisting of related data items of the same type within the consecutive memory.
- ◆ The **elements of an array** can be either **primitive types** or **reference types**.
- ◆ Arrays remain the same length once they are created.
- ◆ Arrays are objects, so they are considered reference types.
- ◆ To refer to a particular element in an array, you specify the name of the reference to the array and the position number of the element in the array.

Arrays in java ... cont'd

- ◆ The position number of the element is called the element's *index* or *subscript*.
- ◆ Each element in an array can be accessed individually using the **name of the array** followed by the **index** or **subscript** of the particular element in the **square brackets []**.
- ◆ The first element in every array has **index zero** and the last element has **index N-1**, if an array has N elements.
- ◆ An index must be an **int** value or a value of a type that can be promoted to **int**—namely, **byte**, **short** or **char**, but not **long**; otherwise, a compilation error occurs.

e.g., **int [] C = new int [10] ;**

0 to 9 are index of the array

C is name of array	6	-24	15	37	40	102	-1	3	65	6
	0	1	2	3	4	5	6	7	8	9

Arrays in java ... cont'd

- ◆ An index must be a **non-negative** integer.
- ◆ Every array object knows its own **length** and stores it in a **length instance variable**.
- ◆ The expression **C.length** accesses array **C**'s length field to determine the length of the array.
- ◆ If you want to compute the sum of the values contained in the first three elements of array **C** and store the result in variable sum.

0 to 9 are index of the array

C is name of array	6	-24	15	37	40	102	-1	3	65	6
	0	1	2	3	4	5	6	7	8	9

e.g., `int sum = C [ 0 ] + C [ 1 ] + C [ 2 ] ; // use the name and index.`

Arrays in java ... cont'd

➤ Declaring and Creating Arrays

- ◆ Array objects occupy space in memory.
- ◆ Like other objects, **arrays** are created with keyword **new**.
- ◆ To create an array object, you specify the type of the array elements and the number of elements as part of an *array-creation expression* that uses keyword **new**.
- ◆ Such an expression returns a reference that can be stored in an array variable.
- ◆ The following *array-creation expression* create an **array object** containing **10 int** elements and store the array's reference in array variable **C**.

```
int [ ] C = new int[10];
```

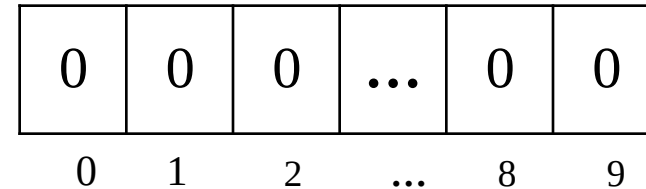

Arrays in java ... cont'd

➤ Creating and Initializing an Array

- ◆ When an array is created, each element of the array receives a default value:

- **Zero** for the numeric primitive-type elements,
- **False** for Boolean elements and
- **Null** for references.

e.g., `int [ ] C = new int [10] ;`



- ◆ In the above assignment statement, the array variable **C** receives the reference to a **new** array of **10 int** elements.
- ◆ A program can create several arrays in a single declaration with comma separated.

For example: `String [ ] b = new String [100], x = new String [27] ;`

Arrays in java ... cont'd

For example:

// Initializing the elements of an array to default values of zero.

```
public class Initialize_Array
{
    public static void main(String[] args)
    {
        int [ ] array; // declare array named array
        array = new int[10]; // create the array object with default value 0

        // output each array element's value
        for( int counter= 0; counter < array.length; counter++)
            System.out.printf ( "%5d%8d\n", counter, array[counter] );

    } // end main
} // end class InitArray
```

Arrays in java ... cont'd

➤ Using an Array Initializer

- ◆ We can create an array and initialize its elements with an array initializer—a comma-separated list of expressions (called an initializer list) enclosed in braces.
- ◆ In this case, the array length is determined by the number of elements in the initializer list.

For example, creating an array with five array elements:

```
int[ ] n = { 10, 20, 30, 40, 50 }; // index values of 0 to 4.
```

- ◆ Element `n[0]` is initialized to 10, `n[1]` is initialized to 20, and so on.
- ◆ When the compiler encounters an array declaration that includes an initializer list, it counts the number of initializers in the list to determine the size of the array.

Arrays in java ... cont'd

For example: // Initializing the elements of an array with an array initializer.

```
public class Initialize_Array {  
    public static void main(String[] args) {  
        // initializer list specifies the value for each element  
        int [ ] array = {32, 27, 64, 18, 95, 14, 90, 70, 60, 37};  
        int total=0;    // initialize total to 0;  
  
        System.out.printf(" %s%8s\n", "Index", "Value"); //column headings  
  
        // output each array element's value  
        for( int counter = 0; counter < array.length; counter++ ){  
            total += array[ counter ] ;    // compute sum  
            System.out.printf(" %5d%8d\n", counter, array[counter]) ;  
        }  
  
        System.out.printf( "Total of array elements: %d\n", total ) ;  
    }  
}
```

Arrays in java ... cont'd

➤ Calculating the Values to Store in an Array:

- ◆ **Constant** variables must be initialized before they are used and cannot be modified thereafter.
- ◆ If you attempt to modify a **final** variable after it's initialized in its declaration, the compiler issues an error message.
- ◆ Constant variables (also are called named constants) often make programs more readable than programs that use literal values.
- ◆ Assigning a value to a constant variable after it has been initialized is a compilation error.
- ◆ Attempting to use a constant before it's initialized is a compilation error.

Example: Arrays in java ... cont'd

// Calculating the values to be placed into the elements of an array.

```
public class InitArray {
```

```
    public static void main( String[] args )
```

```
    {
```

```
        final int Array_length = 10; // declare constant
```

```
        int[ ] array = new int[Array_length ] ; // creating an array
```

```
        // Calculate value for each array element
```

```
        for(int counter = 0; counter < array.length; counter++)
```

```
            array[counter] = 2 + 2 * counter ;
```

```
        System.out.printf( "%6s%8s\n", "Index", "Value" ) ;
```

```
        // Output each array element's value
```

```
        for(int counter = 0; counter < array.length; counter++)
```

```
            System.out.printf(" %6d%8d\n", counter, array[counter]) ;
```

```
    } // end main
```

```
} // end class InitArray
```

Arrays in java ... cont'd

➤ Enhanced for Statement

- ◆ In Java, **enhanced for statement** (also called the "**for-each**" loop) is used to iterate over *arrays* and *collections* such as *lists*, *sets*, or other objects that implement the iterable interface.
- ◆ The **enhanced for statement** iterates through the elements of an array without using a counter, thus avoiding the possibility of "stepping outside" the array.
- ◆ It simplifies the iteration process, making the code more readable and less error-prone.

The syntax of an enhanced for statement is:

```
for (Parameter : ArrayName)  
    statement
```

Arrays in java ... cont'd

➤ Enhanced for Statement

- ◆ The syntax of an enhanced for statement is:

```
for (Parameter : ArrayName)  
    statment
```

- ◆ where *parameter* has a *type* and an *identifier* (e.g., int number), and *arrayName* is the array through which to iterate.
- ◆ *Parameter* is a variable that represents each element of the collection as the loop iterates, and *ArrayName* refers to the array or collection you are iterating over.
- ◆ The type of the parameter must be consistent with the type of the elements in the array.
- ◆ The loop selects one **int** value from the array during each iteration.

Example: Arrays in java ... cont'd

// Using the enhanced for statement to total integers in an array.

```
public class EnhancedForTest {  
  
    public static void main( String[] args ) {  
        int[] array = {87, 68, 94, 100, 83, 78, 85, 91, 76, 87} ;  
        int total = 0 ;  
  
        // for ( int counter = 0; counter < array.length; counter++ )  
        // total += array[ counter ] ;  
  
        // add each element's value to total  
        for ( int number : array )  
            total += number ;  
  
        System.out.printf("Total of array elements: %d%n", total) ;  
  
    } // end main  
} // end class EnhancedForTest
```

Introduction to Files

- **File**

- A computer **file is a collection of data stored** on a **nonvolatile device**.
- Files **exist on permanent storage devices**, such as hard disks, Zip disks, USB drives, reels or cassettes of magnetic tape, and compact discs.

- You can categorize files by the way they store data:

- **Text files**

- Contain **data that can be read in a text editor** because the data has been encoded using a scheme such as ASCII or Unicode.
 - Some text files are data files that contain facts and figures, such as a Student file that contains student id, names, and grades;
 - Some files are program files or application files that store software instructions.
 - are **used only to communicate with humans**

- **Binary files**

- Contain **data that has not been encoded as text**.
 - Their contents are in binary format; the bits represent other types of encoded information, such **as executable instructions or numeric data**
 - These files are **easily read by the computer but not humans**; you cannot understand them by viewing them in a text editor.
 - Examples include images, music, and the compiled program files with a .class extension.
 - Reading and writing binary files is normally done by a program
 - More efficient; computers read and write binary files more easily than text

Binary Vs. Text Files:

- All data and programs are ultimately just zeros and ones
- *Text files*: the bits represent printable characters
 - for example, Java source files are text files
 - so is any file created with a “text editor”
 - Text files are more readable by humans
- *Binary files*: the bits represent other types of encoded information, such as executable instructions or numeric data
 - these files are easily read by the computer but not humans

Java Text Files

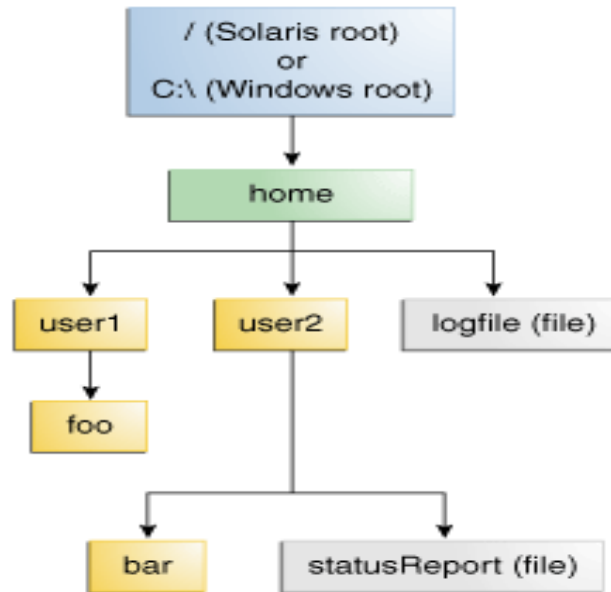
- Source files
- Occasionally input files
- Occasionally output files

Java Binary Files

- Executable files (created by compiling source files)
- Usually input files
- Usually output files

Path

- A file is identified by its path through the file system, beginning from the root node.
- The following figure shows a sample directory tree containing a single root node.



- Microsoft Windows supports multiple root nodes.
- Each root node maps to a volume, such as C:\\ or D:\\.
- The Solaris OS supports a single root node, which is denoted by the slash character, /.

Path...

- The *Path* class, introduced in the Java SE 7 release, is one of the primary entry points of the *java.nio.file* package.
- **Relative or Absolute?**
 - A path is either **relative** or **absolute**.
 - An **absolute path** always contains the root element and the complete directory list required to locate the file.
 - For example, /home/user1/statusReport is an absolute path.
 - All of the information needed to locate the file is contained in the path string.
 - A **relative path** needs to be combined with another path in order to access a file.
 - For example, java/foo is a relative path.
 - Without more information, a program cannot reliably locate the java/foo directory in the file system.

File management

- When you work with stored files in an application, you typically perform the following tasks:
 - Creating Path (file or directory)
 - Checking a File or Directory
 - Determining whether and where a path or file exists
 - Opening a file
 - Writing to a file
 - Reading from a file
 - Closing a file
 - Deleting a file
 - Moving a file
 - Managing Metadata (File and File Store Attributes)
- Java provides built-in classes that contain methods to help you with these tasks.

Using the Path and Files Classes

- **Path class**

- to **create objects that contain information** about files and directories, such as
 - their locations,
 - sizes,
 - creation dates, and
 - whether they even exist.

- **Files class**

- to **perform operations on files and directories**, such as
 - deleting them,
 - determining their attributes, and
 - creating input and output streams.

Using the Path and Files Classes ...

- `import java.nio.file.*;`
 - The `nio` in `java.nio` stands for new input/output
 - The `Path` and `Files` classes are new in Java 7
 - replace the functionality of the **File** class used in older Java versions
 - `Java.io.*`
- **Creating a Path**
- To create a Path,
 - you first determine the default file system on the host computer by using a statement such as:
 - `FileSystem fs = FileSystems.getDefault();`
 - define a Path using the `getPath()` method with it:
 - `Path path = fs.getPath("C:\\programming\\Java\\data.txt");`
 - `FileSystem` method `getSeparator()`.
 - `Path filePath = fs.getPath("C:" + fs.getSeparator() + "programming " + fs.getSeparator() + " Java" + fs.getSeparator() + "data.txt");`
 - Another way to create a Path is to use the `Paths` class (notice the name ends with s).
 - `Path filePath = Paths.get(" C:\\programming\\Java\\data.txt");`

Retrieving Information about a Path

- `String toString()`
 - Returns the String representation of the Path, eliminating double backslashes
- `Path getFileName()`
 - Returns the file or directory denoted by this Path;
 - this is the last item in the sequence of name elements
- `int getNameCount()`
 - Returns the number of name elements in the Path
- `Path getName(int)`
 - Returns the name in the position of the Path specified by the integer parameter
- `Path getParent()`
 - Returns the path of the parent directory.
- `Path getRoot()`
 - Returns the root of the path.
- `Path toAbsolutePath()`
 - Returns the absolute path for a file or directory
- `File toFile()`
 - Returns a file object for the path
- `Boolean isSameFile(p1, p2)`
 - Logic when the paths locate the same file

Statics methods of a Files class

- exists(path)
- notExists(path)
- isReadable(Path)
- isWritable(Path)
- isDirectory(Path)
- isRegularFile(Path)
- isHidden()
- size(Path)
- newDirectoryStream(Path)
- creatFile(Path)
- createDirectory(Path)
- createDirectories(Path)
- delete(Path)
- setReadOnly()
- setLastModified(long time)
- File[] listFiles()

Example: Accessing a Path

```
import java.nio.file.*;
public class PathDemo
{
    public static void main(String[] args)
    {
        Path filePath = Paths.get("C:\\ Programming\\ Java\\Data.txt");
        int count = filePath.getNameCount();
        System.out.println("Path is " + filePath.toString());
        System.out.println("File name is " + filePath.getFileName());
        System.out.println("There are " + count + " elements in the file path");
        for(int x = 0; x < count; ++x)
            System.out.println("Element " + x + " is " + filePath.getName(x));
    }
}
```

Checking File Accessibility

- use the `checkAccess()` method
- import static `java.nio.file.AccessMode.*`
- `filePath.getFileSystem().provider().checkAccess();`
- arguments to the `checkAccess()` method
 - No argument
 - Checks that the file exists; as an alternative,
 - you can substitute the `Files.exists()` method and pass it a `Path` argument.
 - `READ`
 - Checks that the file exists and that the program has permission to read the file
 - `WRITE`
 - Checks that the file exists and that the program has permission to write to the file
 - `EXECUTE`
 - Checks that the file exists and that the program has permission to execute the file
- `filePath.getFileSystem().provider().checkAccess(filePath, READ, EXECUTE);`

Deleting a Path

- Use the Files class delete() method
- Files.delete(filePath);
- The Files class deleteIfExists() method also can be used to delete a file, but if the file does not exist, no exception is thrown.

```
import java.nio.file.*;
import java.io.IOException;
public class PathDemo4
{
    public static void main(String[] args)
    {
        Path filePath =
        Paths.get("C:\\programming\\Java3\\Data.
        txt");
```

```
try
{ Files.delete(filePath)
    System.out.println("File or directory is deleted");
}
catch (NoSuchFileException e) {
    System.out.println("No such file or directory");
}
catch (DirectoryNotEmptyException e) {
    System.out.println("Directory is not empty");
}
catch (SecurityException e) {
    System.out.println("No permission to delete");
}
catch (IOException e) {
    System.out.println("IO exception");
} } }
```

The OpenOptions Parameter

- The following StandardOpenOptions enums are supported:
 - **WRITE** – Opens the file for write access.
 - **APPEND** – Appends the new data to the end of the file. This option is used with the WRITE or CREATE options.
 - **TRUNCATE_EXISTING** – Truncates the file to zero bytes. This option is used with the WRITE option.
 - **CREATE_NEW** – Creates a new file and throws an exception if the file already exists.
 - **CREATE** – Opens the file if it exists or creates a new file if it does not.
 - **DELETE_ON_CLOSE** – Deletes the file when the stream is closed. This option is useful for temporary files.
 - **SPARSE** – Hints that a newly created file will be sparse. This advanced option is honored on some file systems, such as NTFS, where large files with data "gaps" can be stored in a more efficient manner where those empty gaps do not consume disk space.
 - **SYNC** – Keeps the file (both content and metadata) synchronized with the underlying storage device.
 - **DSYNC** – Keeps the file content synchronized with the underlying storage device.

Determining File Attributes

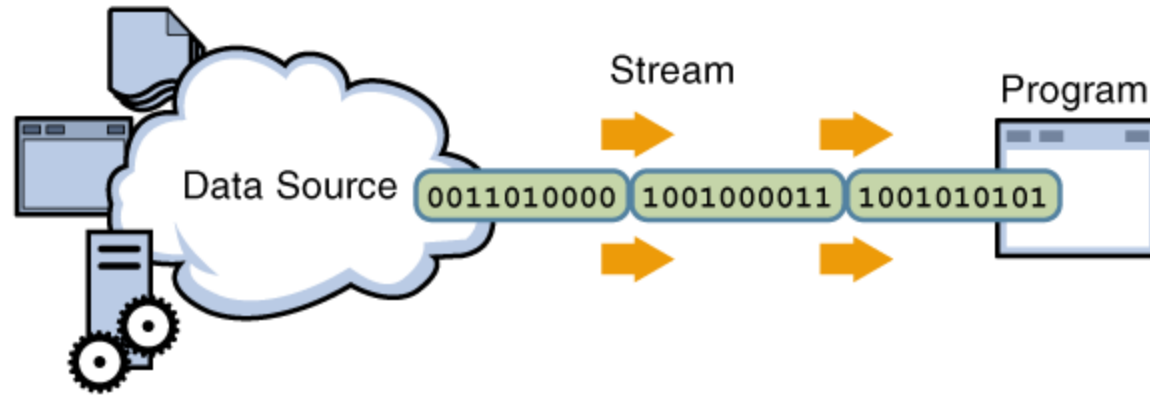
- use the `readAttributes()` method of the `Files` class to retrieve useful information about a file.
- `BasicFileAttributes attr = Files.readAttributes(filePath, BasicFileAttributes.class);`
- Methods such as `creationTime()` and `lastModifiedTime()` return important file times.

```
try
{
    BasicFileAttributes attr = Files.readAttributes(filePath, BasicFileAttributes.class);
    System.out.println("Creation time " + attr.creationTime());
    System.out.println("Last modified time " + attr.lastModifiedTime());
    System.out.println("Size " + attr.size());
}
```

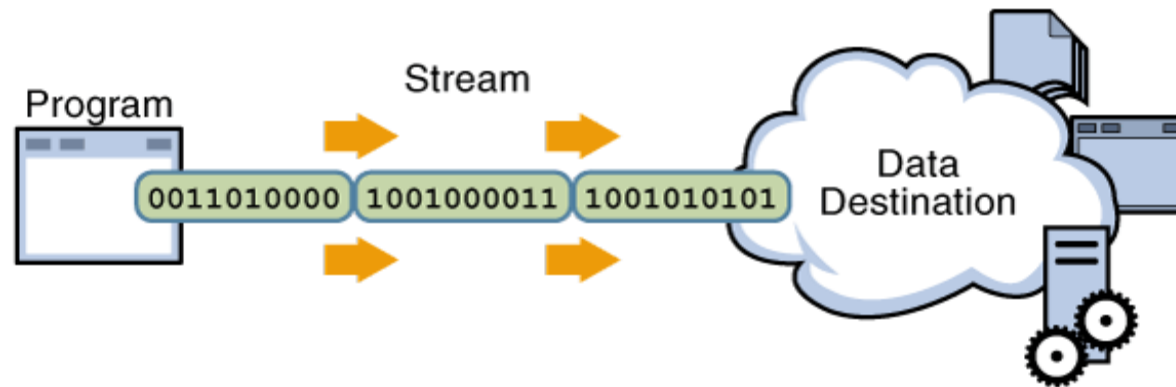
I/O Streams

- A **stream** is simply a **flow of characters to and from a program**.
- The other end of the stream can be anything that can accept or generate a stream of characters, including a console window, a printer, a file on a disk drive, or even another program.
 - **Streams have no idea of the structure or meaning of your data**; a stream is just a sequence of characters.
- An **I/O Stream** represents an **input source** or **an output** destination.
 - A stream can represent many different kinds of sources and destinations, including disk files, devices, other programs, and memory arrays.
- Streams support many different kinds of data, including simple bytes, primitive data types, localized characters, and objects.
 - Some streams simply pass on data; others manipulate and transform the data in useful ways.
- No matter how they work internally, all streams present the same simple model to programs that use them:
 - A stream is a sequence of data.

- **Reading information into a program:** A program uses an input stream to read data from a source, one item at a time.

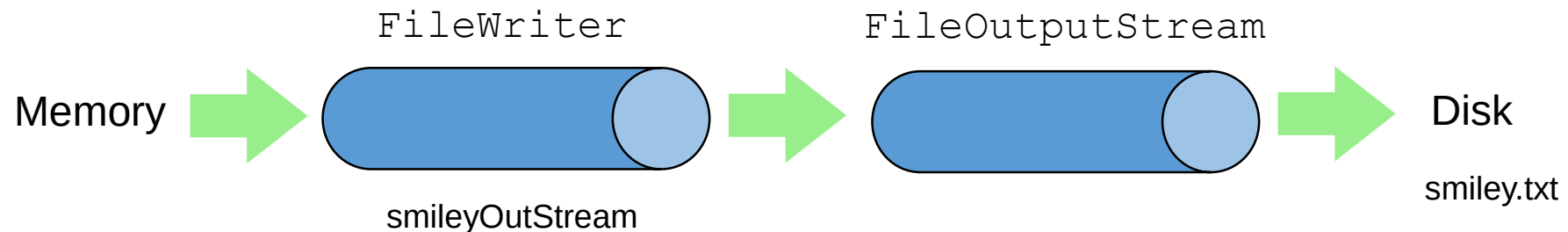


- **Writing information from a program:** A program uses an output stream to write data to a destination, one item at a time.



The java.io Package

- Java programs perform file processing by using classes from package **java.io**
- This package includes definitions for stream classes, such as:
 - ✓ **FileInputStream** (for byte-based input from a file),
 - ✓ **FileOutputStream** (for byte-based output to a file),
 - ✓ **FileReader** (for character-based input from a file) and
 - ✓ **FileWriter** (for character-based output to a file)
- Which inherit from classes `InputStream`, `OutputStream`, `Reader` and `Writer`, respectively.



```
FileWriter smileyOutputStream = new FileWriter( new FileOutputStream("smiley.txt") );
```

- In addition to the java.io classes, character-based input and output can be performed with classes **Scanner** and **Formatter**.
- **Class Scanner** is used extensively **to input** data from the keyboard—it can also **read data from a file**.
- **Class Formatter** enables formatted data to be **output to** any **text-based** stream in a manner similar to method `System.out.printf`.

(Notice: Write a simple code by using the above classes)

Class File

- Class File is useful for **retrieving information** about files or directories from disk.
- Objects of class File **do not** open files or provide any file-processing capabilities.
- However, File objects are used frequently with objects of other java.io classes to specify files or directories to manipulate.

File class Constructors

- `File f = new File(String name);`
- Creates a java File object to represent name of the file or directory in current working directory.
 - `File f = new File(String subdirName, String name);`
- Creates a java File object to represent name of the file or directory present in specified sub directory.
 - `File f = new File(File subdir, String name);`
- **Example 1:** write code to create a File named with “abc.txt” in current working directory?

```
File f = new File("abc.text");  
f.createNewFile();
```

Contd...

- Example 2: write code to create a directory named with “abc123” in current working directory and create a file named with “demo.txt” in that directory?

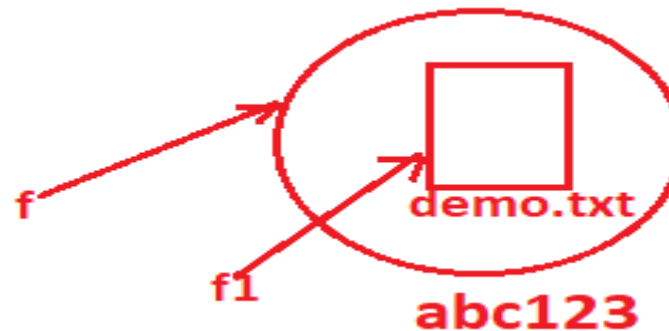
```
File f = new File("abc123");
```

```
f.mkdir();
```

```
//File f1 = new File("abc123","demo.txt");
```

```
File f1 = new File(f, "demo.txt");
```

```
f1.createNewFile();
```



Contd...

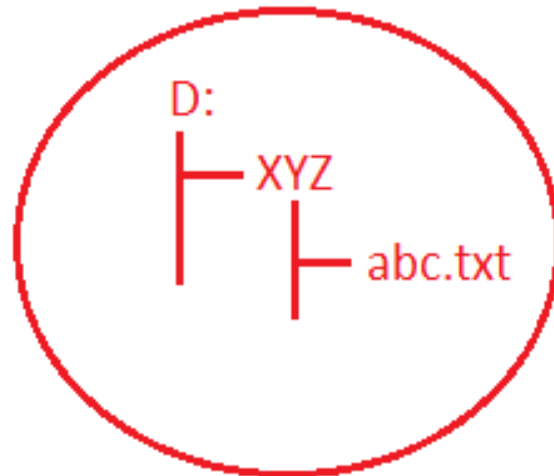
- Example 3: write code to create a file named with “abc.txt” in “D:\\XYZ” folder.

```
//File f = new File("XYZ");
```

```
//f.mkdir();
```

```
File f1 = new File("D:\\XYZ","abc.txt");
```

```
f1.createNewFile();
```



Important methods present in File

- `f.exists()`
- `f.createNewFile()`
- `f.mkdir();`
- `f.isFile()`
- `f.isDirectory()`
- `String [] s = f.list()`
- `long l = f.length()`
- `f.delete()`

Example:

- Write a program to display the names of all files and directories present in “D:\\AdvJavaDemo”?

```
import java.io.*;
class FileDemo3
{
    public static void main(String[] args) throws IOException{
        int count = 0;
        File f = new File("D:\\AdvJavaDemo");
        String [] s = f.list();
        for (String s1:s)
        {
            count++;
            System.out.println(s1);
        }
        System.out.println("The total number of files are:"+count);
    }
}
```

FileWriter class

- We can use **FileWriter** to write character data to the file.

❑ Constructors of **FileWriter**:

1. **FileWriter** fw = new **FileWriter**(**String** fName);
2. **FileWriter** fw = new **FileWriter**(**File** f);

➤ The above **FileWriters** are meant for overriding existing data. instead of overriding, if we want append operation then we have to create **FileWriter** by using the following constructors:

1. **FileWriter** fw = new **FileWriter**(**String** fName, boolean append);
2. **FileWriter** fw = new **FileWriter**(**File** f, boolean append);

Contd...

- **Note:** if the specified file is not already available in all the above constructors will create that file.

□FileWriter methods:

1. `write(int ch)` --- to write a single character to the file.
2. `write(char[] ch)` --- to write an array of characters.
3. `write(String s)` --- to write string to the file.
4. `flush()` --- to give the guarantee that total data including last character will be written to the file.
5. `close()` --- to close the writer

Example:

```
import java.io.*;
class FileWriterDemo1
{
    public static void main(String[] args) throws IOException {
        FileWriter fw = new FileWriter("abc.txt", true);
        fw.write(100); // adding a single character d
        fw.write("ebre\nberhan\nuniversity");
        fw.write('\n');
        char [] ch1 = {'a','b','c'};
        fw.write(ch1);
        fw.flush();
        fw.close();
    }
}
```

abc.txt

debre
berhan
university
abc

FileReader class

- We can use FileReader to read character data from the file.

❑ Constructors:

1. `FileReader fr = new FileReader(String fName);`
2. `FileReader fr = new FileReader(File f);`

❑ Methods:

1. `int read()` --- it attempts to read next character from the file and returns Unicode value. If the next character not available then this method returns -1.
- As this method returns Unicode value(int value), at the time of printing we have to perform type casting.

Example:

```
FileReader fr = new FileReader("abc.txt");  
int i = fr.read();  
while (i != -1)  
{  
    System.out.print((char)i);  
    i = fr.read();  
}
```

2. `int read(char[] ch)` --- it attempts to read enough characters from the file into `char []` and returns number of characters copied from the file.

Example:

```
File f = new File("abc.txt");  
char [] ch = new char [(int)f.length()];  
FileReader fr = new FileReader(f);  
fr.read(ch);  
for (char ch1:ch)  
{  
    System.out.print(ch1);  
}
```

3. void close() --- to close a file

- **Note:** by using FileReader we can read data character by character which is not convenient to the programmer.
- Usage of **FileReader** and **FileWriter** is not recommended because:

Contd...

1. while writing by FileWriter we have to insert line separator (\n) manually which varied from system to system it is difficult to the programmer.
2. By using FileReader we can read data character by character, which is not convenient to the programmer. To overcome these problems we should go for **BufferedWriter** and **BufferedReader**.

BufferedWriter

- We can use BufferedWriter to write character data to the file.

□ Constructors:

1. `BufferedWriter bw = new BufferedWriter(writer w);`
2. `BufferedWriter bw = new BufferedWriter(writer w, int bufferSize);`

- **Note:** BufferedWriter cannot communicate to the file, it can communicate via some writer object.

Q) Which of the following are valid and invalid?

1. `BufferedWriter bw = new BufferedWriter("abc.txt");` --- invalid
2. `BufferedWriter bw = new BufferedWriter(File f, "abc.txt");` --- invalid
3. `BufferedWriter bw = new BufferedWriter(new FW("abc.txt"));` --- valid
4. `BufferedWriter bw = new BufferedWriter(new BW(new FW("abc.txt")));` --- valid

Methods

1. `write(int ch)`
2. `write(char [] ch)`
3. `write(String s)`
4. `flush()`
5. `close()`
6. `newLine()` --- to insert line separator

Q) When compared with `FileWriter` which of the following capability available extra in method form in `BufferedWriter`?

- A. Writing data to the file
- B. Close the file
- C. Flushing the file
- D. Inserting a new line character

Methods ...

- **Writing data to the file:** Both `FileWriter` and `BufferedWriter` can write data to the file.
 - However, `BufferedWriter` just does it more efficiently by buffering the data before writing it to the file.
- **Close the file:** Both `FileWriter` and `BufferedWriter` have a `close()` method to close the file stream.
- **Flushing the file:** `BufferedWriter` provides the `flush()` method, which is not available in `FileWriter`.
 - The `flush()` method ensures that any buffered data is written to the file immediately, without waiting for the buffer to fill up.
- **Inserting a new line character:** `BufferedWriter` has the `newLine()` method to insert a platform-specific new line character.
 - This is an extra capability compared to `FileWriter`, which does not have a method for inserting a new line.

Methods

```
import java.io.*;

class BufferedWriterDemo {

    public static void main(String[] args) throws IOException {

        FileWriter fw = new FileWriter("abc.txt");

        BufferedWriter bw = new BufferedWriter(fw);

        bw.write(100); //adding a single character d

        bw.newLine();

        char [] ch = {'a','b','c','d'};

        bw.write(ch);

        bw.newLine();

        bw.write("Come");

        bw.newLine();

        bw.write("from Debre Birhan");

        bw.flush();

        bw.close(); }}
```

Sequential-Access Text Files

- Java imposes no structure on a file. You must structure files to meet your application's needs.
- To retrieve data sequentially from a file, programs normally start from the beginning of the file and read all the data consecutively until the desired information is found.
- Data in many sequential files cannot be modified without the risk of destroying other data in the file. Records in a sequential-access file are usually updated by rewriting the entire file.

Sequential-Access Text Files ...

- To retrieve data sequentially from a file, programs start from the beginning of the file and read all the data consecutively until the desired information is found.
- It might be necessary to process the file sequentially several times (from the beginning of the file) during the execution of a program.
- Class **Scanner** does *not allow repositioning to the beginning of the file*. If it's necessary to read the file again, the program must close the file and reopen it.

Object Serialization

- Java provides a mechanism called **object serialization** that enables entire objects to be written to a file.
- A **serialized object** is represented as a sequence of bytes that includes the **object's data** as well as **information** about the **object's type** and the **types of data** it stores.
- After a serialized object has been written into a file, it can be read from the file and **deserialized** to recreate the object in memory.
- Classes **ObjectInputStream** and **ObjectOutputStream** enable entire objects to be read from or written to a stream.
- Only classes that implement interface **Serializable** can be serialized and deserialized.

Exercises

1. Write a program to read 1 byte of data from a file?

```
package javaprogrammingdemo;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.*;
public class javalabclass{
    public static void main(String args[]) throws
    IOException{
        try
        {
            FileInputStream fs = new
            FileInputStream("abc.txt");
            int data=fs.read();
```

```
System.out.print((char)data);
        fs.close();
    }
    catch(FileNotFoundException e)
    {
        System.out.println("Check File");
    }
}
}
```


Contd...

2. Write a program to read an Array of Bytes from a File?

```
package javaprogrammingdemo;
import java.io.FileInputStream;
import
java.io.FileNotFoundException;
import java.io.IOException;
import java.util.*;
public class javalabclass{
    public static void main(String
args[]) throws IOException{
    try
    {
        FileInputStream fs = new
FileInputStream("abc.txt");
```

```
        byte[] input = new byte[5];
        fs.read(input);
for(byte b:input)
    {
        System.out.print((char)b);
    }
fs.close();}
    catch(FileNotFoundException e)
    {
        System.out.println("Check
File");
    }}}
```

Contd...

3. Write a program to read all data from a file 1 byte at a time.

```
package javaprogrammingdemo;
import java.io.FileInputStream;
import
java.io.FileNotFoundException;
import java.io.IOException;
import java.util.*;
public class javalabclass{
    public static void main(String
args[]) throws IOException{
        try
        {
            FileInputStream fs = new
FileInputStream("satish.txt");
            int data;
```

```
while((data=fs.read())!=-1)
    {
        System.out.print((char)data);
    }
    fs.close();
    }
    catch(FileNotFoundException
e)
    {
        System.out.println("Check
File");
    }
} }
```

Contd...

4. Write a Java Program to write an array of bytes to a file.

```
package javaprogrammingdemo;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
import java.util.*;
public class javalabclass{
    public static void main(String args[]) throws
IOException{
    try
    {
        //demo of Fileoutputstream
        //file gets created and data is written
        FileOutputStream fout = new
FileOutputStream("output.txt");
```

```
byte[] b = {104,105,106,107};
        fout.write(b);
        fout.close();
        System.out.println("File Write
Successful");
    }
    catch(FileNotFoundException e)
    {
        System.out.println("Check File");
    }
    catch(Exception e)
    {
        System.out.println("Exception
Occurred");
        e.printStackTrace();
    } } }
```

Example 1: Create and Write to a file by using File and PrintWriter classes

```
package fileio;

import java.io.File;
import java.io.PrintWriter;
import java.io.IOException;

public class tryfileio {

    public static void main(String[] args) {
        File file = new File("test.txt");
        try {
            PrintWriter output = new PrintWriter(file);

            output.println("G3Comp");
            output.println("A,B & C");
            output.close();
        } catch (IOException ex) {
            System.out.printf("ERROR:%s\n", ex);
        }
    }
}
```

Example 2: Read from a file by using Scanner class

```
package fileio;

import java.io.File;
import java.util.Scanner;
import java.io.IOException;

public class tryfileio {

    public static void main(String[] args) {
        File file = new File("test.txt");
        try {
            Scanner read = new Scanner(file);

            String name1=read.nextLine();

            String name2=read.nextLine();
            read.close();

            System.out.print(name1);

            System.out.print(name2);
        } catch (IOException ex) {
            System.out.printf("ERROR:%s\n", ex);
        }
    }
}
```

Example 3: Write to a file by using FileWriter class

```
package fileprocess;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;

public class FileWritingDemo {

    public static void main(String[] args) {
        File myfile = new File("test1.txt");
        try {
            FileWriter fwrite = new FileWriter(myfile);

            fwrite.write("Hi java learners!!!");
            fwrite.flush();
            fwrite.close();
        } catch (IOException ext) {
            System.out.printf("Error : %s", ext);
        }
    }
}
```

Notice:

There are different java classes for read and write operations. It is expected from you to practice creating file, writing to and reading from the file by applying some of these classes.

Database Connectivity

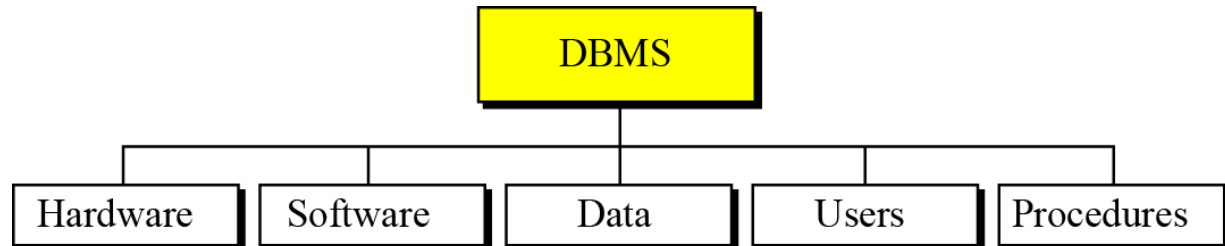
- JDBC programming concept
- populating database
- executing queries
- Manipulating query results

Database System

- A database is
 - a **collection of related and logically coherent data** used by the application programs in an organization.
- Advantages of databases
 - Less redundancy
 - Inconsistency avoidance
 - Efficiency
 - Data integrity
 - Confidentiality

DBMS

- A database management system (DBMS)
 - defines,
 - creates and
 - maintains a database.
 - allows controlled access to data in the database.
- A DBMS is a combination of five components:
 - hardware,
 - software,
 - data,
 - users and
 - procedures



Relational Database Model

- Data are organized in two-dimensional tables called relations.
- The tables are related to each other.
- A relation in an RDBMS has the following features:
 - Name
 - Attributes
 - Tuples

Relational Integrity Constraints

- Constraints are **conditions** that must hold on **all** valid relation states.
- There are three *main types* of constraints in the relational model:
 - **Key** constraints
 - **Entity integrity** constraints
 - **Referential integrity** constraints
- Another implicit constraint is the **domain** constraint
 - Every value in a tuple must be from the *domain of its attribute* (or it could be **null**, if allowed for that attribute)

Introduction Structured query language

- What is SQL?

- It is a programming language for storing and processing information in a relational database.
- A relational database stores information in tabular form, with rows and columns representing different data attributes and the various relationships between the data values.
- You can use SQL statements to store, update, remove, search, and retrieve information from the database.
- You can also use SQL to maintain and optimize database performance.

- Why is SQL important?

- SQL is a popular query language that is frequently used in all types of applications.
- Data analysts and developers learn and use SQL because it integrates well with different programming languages.
 - For example, they can embed SQL queries with the Java programming language to build high-performing data processing applications with major SQL database systems such as Oracle or MS SQL Server.
 - SQL is also fairly easy to learn as it uses common English keywords in its statements

Types of SQL Statements

- SQL can be separated into four types of statements:
 - Data Definition Language (DDL)
 - Data Control Language (DCL)
 - Data Manipulation Language
 - Data query language (DQL)
- **Data Definition Language (DDL)** statements are used to **create**, **modify**, or **delete** database objects such as **tables**, **views**, **schemas**, **domains**, **triggers**, **stored procedures**, and the database itself.
 - The SQL keywords most often associated with **DDL statements** are **CREATE**, **ALTER**, and **DROP**.
 - For example, you would use the CREATE TABLE statement to create a table, the ALTER TABLE statement to modify the table's properties, and the DROP TABLE statement to delete the table definition from the database.

Types of SQL Statements ...

- **Data Control Language (DCL)** protects your database from becoming corrupted.
 - Used correctly, the DCL provides security for your database; the amount of protection depends on the implementation.
 - If your implementation doesn't provide sufficient protection, you must add that protection to your application program.
 - DCL statements **allow you to control who or what** (a database user can be a person or an application program) has access to specific objects in your database.
 - With DCL, **you can grant or restrict access** by using the **GRANT**, **Deny** or **REVOKE** statements, the three primary DCL commands.
 - The DCL statements also allow you to control the type of access each user has to database objects.

Types of SQL Statements ...

- **Data Manipulation Language (DML)** allows a user or an application program to **update the database by adding new data, removing** old data, and **modifying** previously stored data.
 - The primary keywords associated with **DML statements are INSERT, UPDATE, and DELETE**, all of which represent the types of statements you'll probably be using the most.
 - For example, you can use a update statement to modify existing data from a table and an insert statement to add data to a table.
- **Data query language (DQL)** When you need to **retrieve data from a database**, you use the SQL language to make the request.
 - The DBMS processes the SQL request, retrieves the requested data, and returns it to you.
 - This process of requesting data from a database and receiving back the results is called a database query—hence the name Structured Query Language.
 - You use a SELECT statement to retrieve data from a table.

SQL ...

- In a relational database, we can define several operations to create new relations out of the existing ones.
- Basic operations:
 - Insert
 - Delete
 - Update
 - Select
 - Project
 - Join
 - Union
 - Intersection
 - Difference

Possible violations for each operation

- INSERT may violate any of the constraints:
 - Domain constraint:
 - if one of the attribute values provided for the new tuple is not of the specified attribute domain
 - Key constraint:
 - if the value of a key attribute in the new tuple already exists in another tuple in the relation
 - Referential integrity:
 - if a foreign key value in the new tuple references a primary key value that does not exist in the referenced relation
 - Entity integrity:
 - if the primary key value is null in the new tuple

Violations ...

- DELETE may violate only referential integrity:
 - If the primary key value of the tuple being deleted is referenced from other tuples in the database
 - Can be remedied by several actions: RESTRICT, CASCADE, SET NULL
 - RESTRICT option: reject the deletion
 - CASCADE option: propagate the new primary key value into the foreign keys of the referencing tuples
 - SET NULL option: set the foreign keys of the referencing tuples to NULL
 - One of the above options must be specified during database design for each foreign key constraint

Violations ...

- UPDATE may violate domain constraint and NOT NULL constraint on an attribute being modified
- Any of the other constraints may also be violated, depending on the attribute being updated:
 - Updating the primary key (PK):
 - Similar to a DELETE followed by an INSERT
 - Need to specify similar options to DELETE
 - Updating a foreign key (FK):
 - May violate referential integrity
 - Updating an ordinary attribute (neither PK nor FK):
 - Can only violate domain constraints

Relational Database Management System

- Networked RDBMS
 - Oracle
 - For large mission critical systems runs on Linux
 - MS SQL Server
 - Used in small to medium sized systems run in Windows server
 - DB2
 - For large mission critical systems runs on IBM OS
 - MySQL
 - Open source, runs on major operating system, commonly used in web application
 - Apache Derby/Java DB
 - New java based relation database, can be embedded in application
 - ...
- Embedded RDBMS
 - Apache Derby/Java DB
 - MS Access
 - Compact/SQLite version of MS Sql, Oracle

Java Database Connectivity

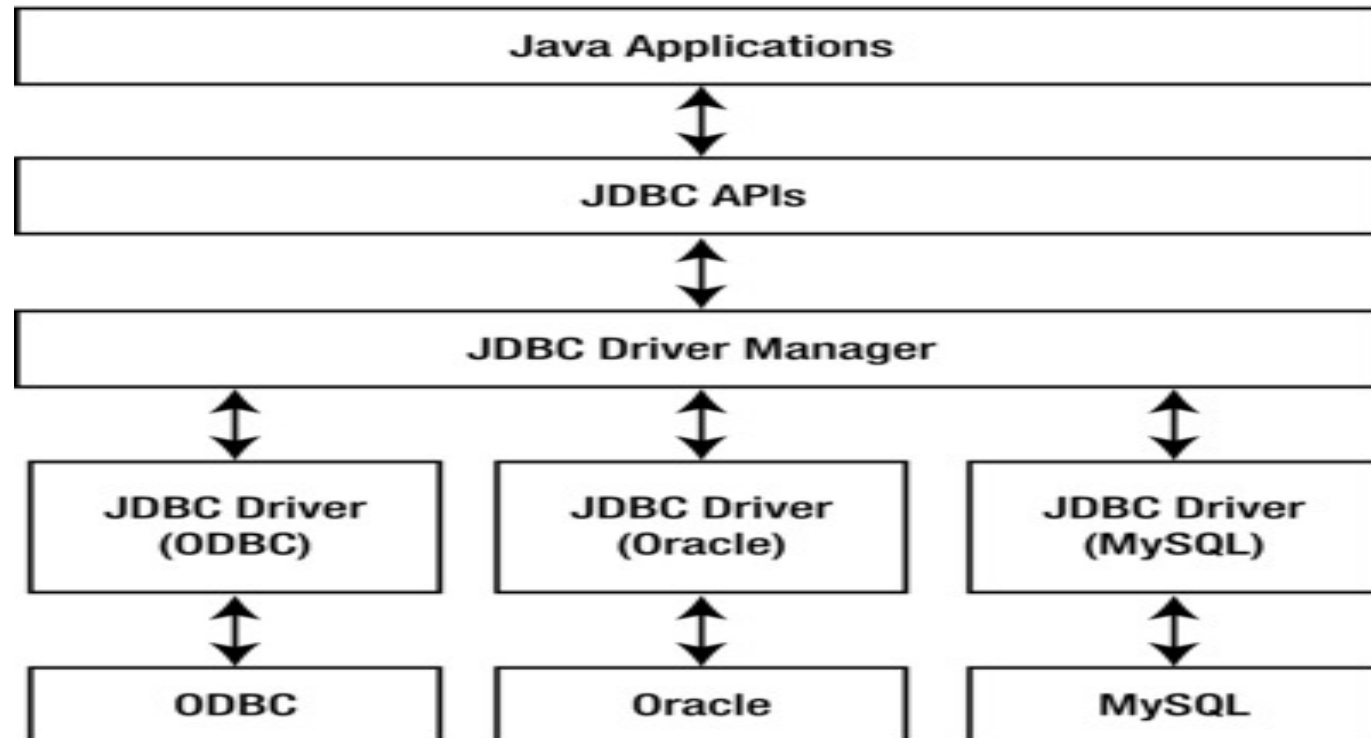
- An API that lets you access virtually any tabular data source from the Java programming language
- is a Java feature that lets you
 - connect to almost any relational database system,
 - execute SQL commands, and
 - process the results all from within a Java program
- JDBC provides Java applications with access to most database systems via SQL
- ODBC is a generic database connection standard that almost every database program available can speak to.
- It's inherently inefficient, but it is easy to set up and performs adequately for small applications and for testing purposes. E.g. Java to MS Access

Java Database Connectivity

- In its simplest form, JDBC makes it possible to do the following:-
 - Connect to a database
 - Execute SQL statements to query your database
 - Generate query results
 - Perform updates, inserts and deletions
 - Execute Stored Procedures
- The benefits of using JDBC include the following:-
 - A developer only needs to write one API to access any relational database
 - There is no need to rewrite code for different databases
 - There is no need to know the database vendor's specific APIs
 - It provides a standard API and is vendor independent
 - Almost every database vendor has some sort of JDBC driver

JDBC Architecture

- The JDBC architecture consists of two layers:
 - **first**, the JDBC API, which supports Java application-to-JDBC Driver Manager communications and
 - **secondly** the JDBC Driver API, which handles Driver Manager-to-Database communications.

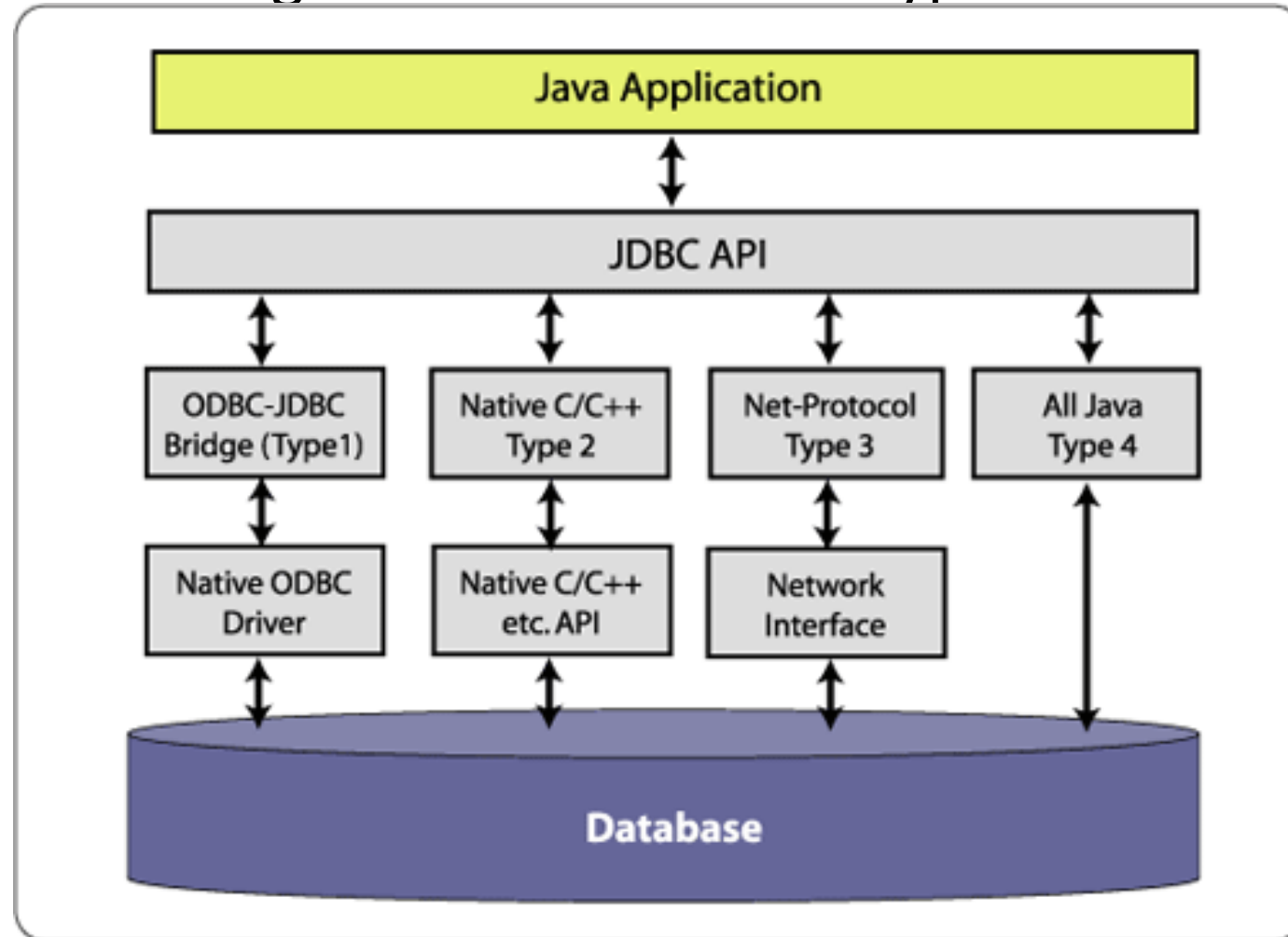


JDBC drivers

- JDBC drivers are available for most database platforms and from a number of different vendors.
- There are four different driver types as follows:
 - Type 1 - JDBC-ODBC Bridge
 - Type 2 - JDBC-Native Bridge
 - Type 3 - JDBC-Net Bridge
 - Type 4 - Direct JDBC Driver
- Type 1 only runs on platforms where ODBC is available
 - ODBC must be configured separately
- Type 2 Drivers map between a proprietary Database API and the JDBC API
- Type 3 Drivers are used with middleware products
- Type 4 Drivers are written in Java
- In most cases, type 4 drivers are preferred

JDBC drivers ...

- The following diagram shows the JDBC architecture options using the different driver types:

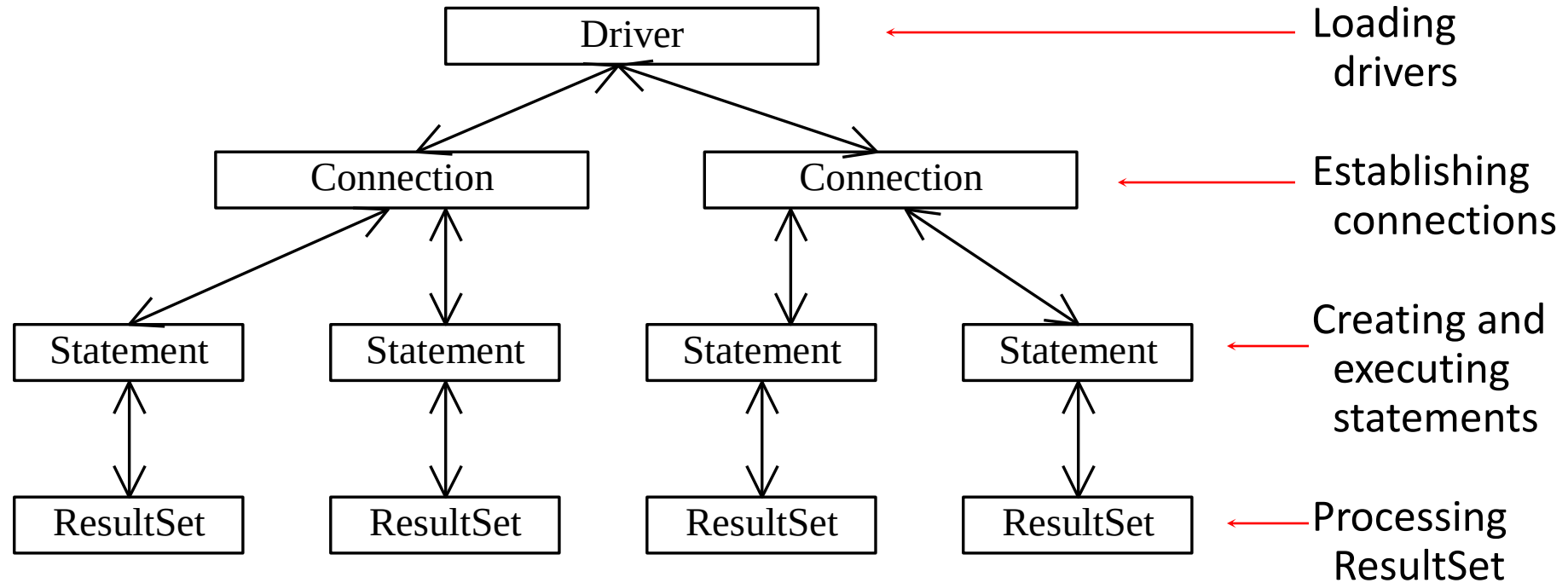


Connecting to a Database

Steps to using a database query:

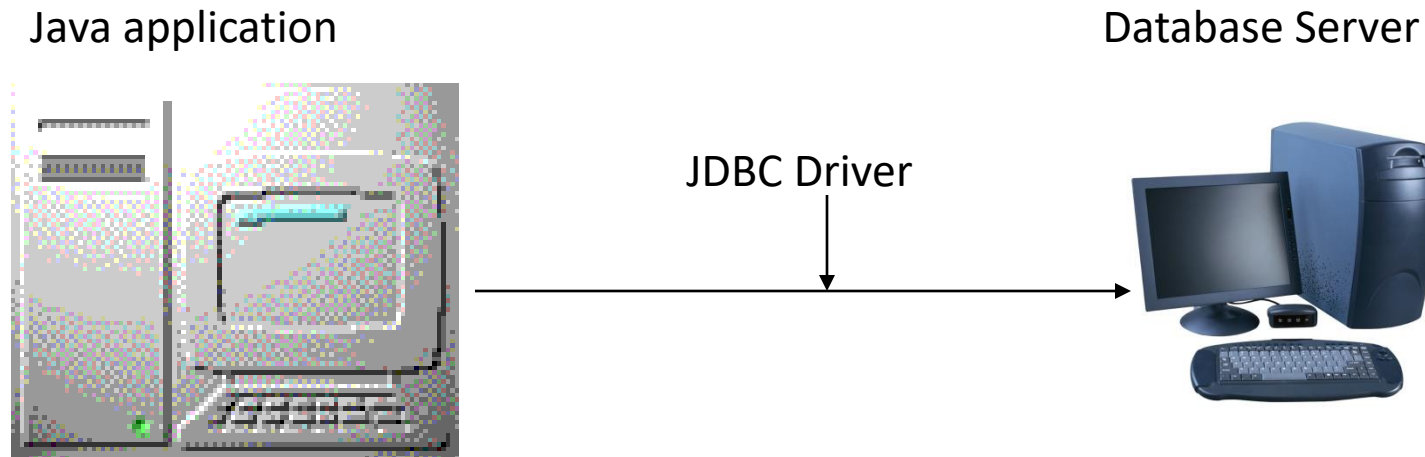
1. Load a JDBC “driver”
2. Connect to the data source
3. Send/execute SQL statements
4. Process the results
5. Close/dispose connection/objects

The JDBC Interfaces



JDBC Driver

- Acts as the gateway to a database
- Not actually a “driver”, just a .jar file



JDBC Driver Installation

- Must download the driver, copy and add the .jar file to your CLASSPATH or project directory
- Download the MySQL JDBC “driver” (Connector/J) from www.mysql.org and copy the .jar file to your project directory or classpath

JDBC Driver Management

- All drivers are managed by the DriverManager class
- Example - loading an Oracle JDBC driver:
 - In the Java code:
Class.forName(“oracle.jdbc.driver.OracleDriver”)
- Driver class names:
 - Oracle: oracle.jdbc.driver.OracleDriver
 - MySQL: com.mysql.jdbc.Driver
 - MS SQL Server:
com.microsoft.jdbc.sqlserver.SQLServerDriver
 - MS Access
sun.jdbc.odbc.JdbcOdbcDriver Already in JDK

Developing JDBC Programs

Loading drivers

```
Connection connection = DriverManager.getConnection(databaseURL);
```

Establishing connections

Database	URL Pattern
Access	jdbc:odbc:dataSource
MySQL	jdbc:mysql://hostname:port/dbname?user=<uname>&password=<password>
Oracle	jdbc:oracle:thin:@hostname:port#:oracleDBSID

Creating and executing statements

Processing ResultSet

Examples:

For Access:

```
Connection connection = DriverManager.getConnection  
("jdbc:odbc:ExampleMDBDataSource");
```

For MySQL:

```
Connection connection = DriverManager.getConnection  
("jdbc:mysql://localhost/test?user=root&password=1234");
```

For Oracle:

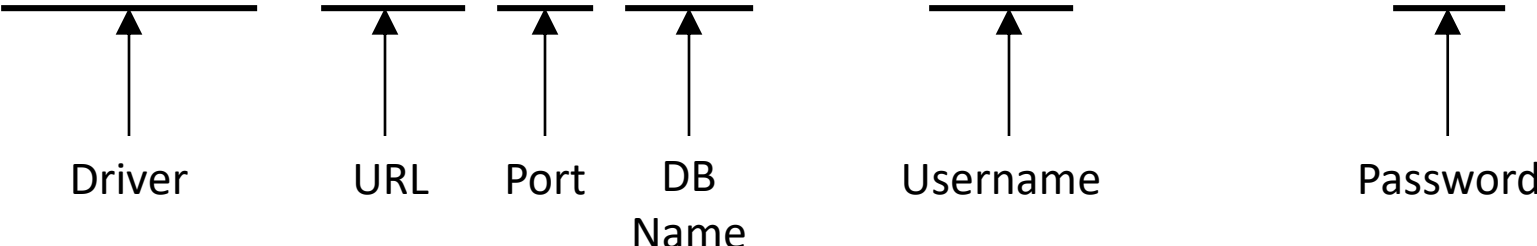
```
Connection connection = DriverManager.getConnection  
("jdbc:oracle:thin:@localhostu:1521:orcl", "scott", "tiger");
```


Establishing a Connection

- Create a Connection object
- Use the DriverManager to grab a connection with the getConnection method
- Necessary to follow exact connection syntax
 - the parameter syntax for getConnection varies between JDBC drivers
 - one driver can have several different legal syntaxes

Establishing a Connection (cont.)

MySQL Example

- Connection con =
DriverManager.getConnection(string);
- what to supply for string ?
- “jdbc:mysql://<URL>:3306/<DB>?user=<user>&password=<pw>”


The diagram shows the string "jdbc:mysql://<URL>:3306/<DB>?user=<user>&password=<pw>" with horizontal lines under the following parts: "jdbc:mysql:", "<URL>", "3306/", "<DB>", "<user>", and "<pw>". Arrows point from these underlined parts to labels below: "Driver" (under "jdbc:mysql:"), "URL" (under "<URL>"), "Port" (under "3306/"), "DB Name" (under "<DB>"), "Username" (under "<user>"), and "Password" (under "<pw>").

Connecting to a database

```
private static Connection getConnection()
{
    Connection con = null;
    try
    {
        //Class.forName("com.mysql.jdbc.Driver");
        String url = "jdbc:mysql://localhost/SISDB";
        String user = "root";
        String pw = "1234";
        con = DriverManager.getConnection(url, user, pw);
    }
    catch (ClassNotFoundException e)
    {
        System.out.println(e.getMessage());
        System.exit(0);
    }
    catch (SQLException e)
    {
        System.out.println(e.getMessage());
        System.exit(0);
    }
    return con;
}
```

Querying a Database

- Connection
 - the Connection class has two methods
 - Close method closes the connection, and
 - The createStatement method returns a Statement object,
- Statement
 - Used for executing SQL statements against the database
 - executeQuery method
 - executeUpdate method
 - execute method
- Obtain a statement object from the connection:
 - Statement stmt = con.createStatement ();
- Execute the SQL statements:
 - stmt.executeUpdate("update table set field='value'");
 - stmt.executeUpdate("INSERT INTO mytable VALUES (1, 'name')");
 - stmt.executeQuery("SELECT * FROM mytable");
- PreparedStatement
 - An SQL statement (which can contain parameters) is compiled and stored in the database
- CallableStatement
 - Used for executing stored procedures

Connection and Statement Methods

- `void close()`
- `statement createStatement()`
- `Statement createStatement(int type, int concur)`
- `ResultSet executeQuery(String sql)`
- `ResultSet executeQuery(String sql)`
- `int executeUpdate(String sql)`
- Parameter of the `createStatement` method
 - The first specifies the type of result set that is created, and can be one of the following:
 - `ResultSet.TYPE_FORWARD_ONLY`
 - `ResultSet.TYPE_SCROLL_INSENSITIVE`
 - `ResultSet.TYPE_SCROLL_SENSITIVE`
 - The second indicates whether the result set is read-only or updatable, and can be one of the following:
 - `ResultSet.CONCUR_READ_ONLY`
 - `ResultSet.CONCUR_UPDATABLE`

Retrieving Data

- ResultSet
 - Represents the result of an SQL statement; rows returned from a query
 - provides methods you can use to move from row to row and to get the data for a column.
- DatabaseMetaData
 - Provides access to a database's system catalogue
- ResultSetMetaData
 - Provides information about the data contained within a ResultSet
- `ResultSet rs = stmt.executeQuery(“SELECT id,name FROM employees where id = 100”)`
 - Some methods used in ResultSet:
 - `next()`
 - `getString()`
 - `getInt()`

Navigation Methods of the ResultSet

- void close()
- void last()
- int getRow()
- boolean next()
- void last()
- boolean isLast()
- boolean isFirst()
- void previous()
- void afterLast()
- void beforeFirst()
- boolean isBeforeFirst()
- boolean isAfterLast()
- absolute(int row)
- relative(int row)

Get Methods of the ResultSet

- `boolean getBoolean(String columnName)`
- `boolean getBoolean(int columnIndex)`
- `Date getDate(String columnName)`
- `Date getDate(int columnIndex)`
- `double getDouble(String columnName)`
- `double getDouble(int columnIndex)`
- `float getFloat(String columnName)`
- `float getFloat(int columnIndex)`
- `int getInt(String columnName)`
- `int getInt(int columnIndex)`
- `long getLong(String columnName)`
- `long getLong(int columnIndex)`
- `short getShort(String columnName)`
- `short getShort(int columnIndex)`
- `String getString(String columnName)`
- `String getString(int columnIndex)`

Working with prepared statement

- PreparedStatement
 - An SQL statement (which can contain parameters) is compiled and stored in the database
- To use it
 - Includes a placeholder(?) in the statement for parameters whose values vary, then
 - Before execute the statement, set the values of those parameters
 - It is a 1-based index
- Provide two benefits
 - They execute faster than the regular statements, b/c db can cache and reuse prepared statements
 - More secure, they prevent SQL injection attacks
- E.g.
 - `String sql = "Insert into course(code, title, credit) values(?,?,?)"`
 - `PreparedStatement pst = conn.prepareStatement(sql);`
 - `pst.setString(1, "CoSc2084");`
 - `pst.setString(2, "Advanced Programming with JAVA");`
 - `pst.setInt(3, "6");`
 - `int arow=pst.excuteUpdate();`

Scrollable and Updatable Result Sets

- The most useful improvements in JDBC 2 are in the `ResultSet` class
- Until the release of JDBC2.1 the virtual cursor could only be moved down the `ResultSet` object.
- But now the virtual cursor can be moved backwards or even positioned at a specific row.
- Also we can modify the values of resultset.
- To obtain scrolling result sets from your queries, you must obtain a different `Statement` object with the method
`Statement stat = conn.createStatement(type, concurrency);`

Cont'd.....

- For a prepared statement, use the call
`PreparedStatement stat = conn.prepareStatement(command,type, concurrency);`
- **ResultSet type values:**
 - `TYPE_FORWARD_ONLY`
 - The result set is not scrollable.
 - `TYPE_SCROLL_INSENSITIVE`
 - The result set is scrollable but not sensitive to database changes.
 - does not reflect changes made to the underlying data source while it is open, but reflects it later
 - `TYPE_SCROLL_SENSITIVE`
 - The result set is scrollable and sensitive to database changes.
- **ResultSet concurrency values**
 - `CONCUR_READ_ONLY`
 - The result set cannot be used to update the database.
 - `CONCUR_UPDATABLE`
 - The result set can be used to update the database.

Cont'd.....

- Three ways in which a ResultSet can be changed, by
 - updating value of a row,
 - deleting a row and
 - inserting a new row.
- Once we get the ResultSet the updateXXX() method is used to change the value of a row of ResultSet.
- This method takes two arguments,
 - first either the name or number of the column of the ResultSet that is to be updated and
 - the second is the new value that will replace the old.
- The value can be set to null by calling updateNull() method.
- It requires one parameter which is number of the column in the current row of the ResultSet.
- The updateRow() method is called after updateXXX() method.
- It changes the values of the column of current row based on the values of updateXXX() method.
- The deleteRow() method is used to delete a row.
- To insert a new row moveToInsertRow() method is there.
- Then set all the fields by updateXXX() method.
- After that insertRow() should be called to insert the row.

Cont'd.....

- There are other methods of ResultSet object that are used to position the virtual cursor in addition to next().
- Those are
 - first()- moves the cursor to first row.
 - last()- moves the cursor to the last row.
 - previous()-moves the cursor to the previous row.
 - absolute()-moves to a specific row as the parameter passed
 - relative()-moves to a row relative to another row
 - getRow()-returns the current row number.
 - afterLast() - Moves the cursor to the end of this ResultSet object, just after the last row
 - beforeFirst() - Moves the cursor to the front of this ResultSet object, just before the first row
 - cancelRowUpdates() - Cancels the updates made to the current row in this ResultSet object
 - moveToCurrentRow() - Moves the cursor to the remembered cursor position, usually the current row
 - refreshRow() - Refreshes the current row with its most recent value in the database

Cont'd.....

- Find whether ResultSet is scrollable:
- Some JDBC drivers may not support all these features and therefore they are unable to return a scrollable ResultSet.
- To know about it,
boolean forward,sensitive,insensitive;
Forward=DatabaseMetaData.supportsResultSetType(TYPE_FORWARD_ONLY);
sensitive=DatabaseMetaData.supportsResultSetType(TYPE_SCROLL_SENSITIVE);
Forward=DatabaseMetaData.supportsResultSetType(TYPE_SCROLL_INSENSITIVE);

Example: Updatable Result Set

```
private void readFaculty() {  
    DefaultTableModel facultyTblModel = (DefaultTableModel) tblFacultyList.getModel();  
    facultyTblModel.setRowCount(0);  
    String sql = "Select * From Faculty order by FacultyName ASC";  
    try {  
        conn = ConnectToDb.getConn();  
        stmt = conn.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE);  
        rst = stmt.executeQuery(sql);  
        int row = 0;  
        while (rst.next()) {  
            facultyTblModel.addRow(new Object[]{rst.getInt("FacultyCode"), rst.getString("FacultyName"), rst.getString("FacultyPhone"),  
rst.getString("FacultyFax"), rst.getString("FacultyBuilding"), rst.getString("FacultyOfficeNumber")});  
        }  
    } catch (SQLException e) {  
        JOptionPane.showMessageDialog(this, "Database server error" + e.getMessage());  
    }  
}
```

Example

Code	Faculty	Phone	Fax	Building	Office No.
20	College of Agricultural Sciences	(251)-912654899	(251)-912654812	B-402	G001
21	College of Art and theatrical Sciences	(251)-912654845	(251)-912654845	B-25	105
1	College of Computing Science	(251)-912654812	(251)-912654894	B-506	G001
11	College of Engineering	(251)-912654845	(251)-912654812	B852	G003
3	College of Enviromental Science	(251)-912654899	(251)-912654812	B-402	G001
2	College of Natural Sciences	(251)-912654841	(251)-912654842	B-403	G001
9	College of Social Science	(251)-912111111	(251)-912111110	B-78	201
10	Faculty of Bussines and Economics	(251)-912654865	(251)-912654812	B-78	G002
4	School of Medicine and Health Science	(251)-912654845	(251)-912654841	B-101	301
	College of Education and				

Save

Delete

Update

Refresh

Cancel

Example: Insert

```
private void insertFaculty() {
    DefaultTableModel facultyTblModel =
        (DefaultTableModel) tblFacultyList.getModel();
    if
        (btnNew.getText().equalsIgnoreCase("New")) {
        int emptrow =
            facultyTblModel.getRowCount() + 1;
        facultyTblModel.setRowCount(emptrow);
        btnNew.setText("Save");
    } else {
        int newrow =
            facultyTblModel.getRowCount()-1;
        try {
            rst.moveToInsertRow();

            rst.updateString("FacultyName",
                facultyTblModel.getValueAt(newrow,
                    1).toString());

            rst.updateString("FacultyPhone",
                facultyTblModel.getValueAt(newrow,
                    2).toString());
```

```
            rst.updateString("FacultyFax",
                facultyTblModel.getValueAt(newrow,
                    3).toString());

            rst.updateString("FacultyBuilding",
                facultyTblModel.getValueAt(newrow,
                    4).toString());

            rst.updateString("FacultyOfficeNumber",
                facultyTblModel.getValueAt(newrow,
                    5).toString());

            rst.insertRow();
            rst.moveToCurrentRow();

            JOptionPane.showMessageDialog(this,
                "Record saved successfully.");
            btnNew.setText("New");
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this,
                "Database server error" + e.getMessage());
        }
    }
}
```

Example: Update

```
private void updateFaculty() {
    DefaultTableModel facultyTblModel = (DefaultTableModel) tblFacultyList.getModel();
    try {
        int resp = JOptionPane.showConfirmDialog(this, "Are you sure you want to update the selected record?");
        if (resp == JOptionPane.YES_OPTION) {
            int row = tblFacultyList.getSelectedRow() + 1;
            rst.absolute(row);
            rst.updateString("FacultyName", facultyTblModel.getValueAt(tblFacultyList.getSelectedRow(), 1).toString());
            rst.updateString("FacultyPhone", facultyTblModel.getValueAt(tblFacultyList.getSelectedRow(), 2).toString());
            rst.updateString("FacultyFax", facultyTblModel.getValueAt(tblFacultyList.getSelectedRow(), 3).toString());
            rst.updateString("FacultyBuilding", facultyTblModel.getValueAt(tblFacultyList.getSelectedRow(), 4).toString());
            rst.updateString("FacultyOfficeNumber", facultyTblModel.getValueAt(tblFacultyList.getSelectedRow(), 5).toString());
            rst.updateRow();
            JOptionPane.showMessageDialog(this, "Record updated successfully.");
        }
    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "Database server error" + e.getMessage());
    }
}
```

Example: Delete

```
private void deleteFaculty() {  
    try {  
        int resp = JOptionPane.showConfirmDialog(this, "Are you sure you want to delete  
the selected record?");  
        if (resp == JOptionPane.YES_OPTION) {  
            int row = tblFacultyList.getSelectedRow() + 1;  
            rst.absolute(row);  
            rst.deleteRow();  
            JOptionPane.showMessageDialog(this, "Record deleted successfully.");  
        }  
    } catch (Exception e) {  
        JOptionPane.showMessageDialog(this, "Database server error" + e.getMessage());  
    }  
}
```

- Reading Assignment
 - CallableStatement
 - How to use CallableStatement to executing stored procedures

End of Chapter

Thank you !